

1. Introduzione

Il progetto prevede una suite di test unitari realizzata con JUnit 4 per verificare il funzionamento delle classi del backend (come **Address**, **CartItem**, **Model**, **Order**, **Product**, **SoldProduct**, **User**, **Cart**, **Current**) e della classe **DatabaseManager**. L'obiettivo è assicurarsi che ogni metodo (inclusi costruttori, getter, setter e la logica interna) si comporti correttamente, gestendo anche i casi limite e le eccezioni. Va inoltre sottolineato che, oltre ai test automatici, abbiamo svolto un accurato testing manuale durante lo sviluppo. Questo processo molto dispendioso ha permesso di identificare e risolvere situazioni e anomalie che spesso sfuggono ai test automatizzati, garantendo così un elevato livello di affidabilità del sistema. I test automatici sono stati implementati solo al termine della stesura del codice, con l'obiettivo di verificare in maniera sistematica e ripetibile il corretto funzionamento della logica applicativa. Questa strategia ha richiesto impegno, ma ha assicurato che tutte le funzionalità critiche siano state messe alla prova prima della consegna finale.

2. Panoramica dei Test

La strategia di testing adottata include:

- **Test dei costruttori:** Verifica dell'inizializzazione corretta degli oggetti.
 - **Test dei getter e setter:** Conferma che gli attributi possano essere impostati e recuperati correttamente.
 - **Test della logica di business:** Validazione di comportamenti particolari, come l'aggiunta e la rimozione di elementi dal carrello e l'aggiornamento delle quantità.
 - **Test dei metodi statici:** Verifica della gestione dello stato globale (ad es. nella classe **Current**).
 - **Test dell'accesso al database:** La classe **DatabaseManager** è stata sottoposta a test per garantire il corretto funzionamento delle operazioni CRUD (create, read, update, delete) e delle query, assicurando che le connessioni e le risorse vengano gestite correttamente.
-

3. Dettagli dei Test per Classe

3.1. AddressTest

- **Obiettivo:** Assicurare che i costruttori e i metodi setter/getter funzionino correttamente.

- **Casi di test:**
 - Inizializzazione tramite costruttore di default e impostazione tramite setter.
 - Verifica del costruttore parametrizzato e della coerenza dei dati.

3.2. CartItemTest

- **Obiettivo:** Validare che l'oggetto **CartItem** venga creato correttamente e che i metodi getter/setter restituiscano i valori attesi.
- **Casi di test:**
 - Inizializzazione tramite costruttore e verifica dei valori.
 - Modifica del prodotto e della quantità tramite setter.

3.3. ModelTest

- **Obiettivo:** Testare le diverse modalità di creazione di un **Model** (inclusi costruttori con e senza ID o immagine) e il corretto funzionamento dei relativi getter/setter.
- **Casi di test:**
 - Verifica della creazione e della lettura dei dati per ogni costruttore implementato.

3.4. OrderTest

- **Obiettivo:** Verificare che la classe **Order** venga creata correttamente, soprattutto in relazione al recupero degli item venduti tramite il **DatabaseManager**.
- **Casi di test:**
 - Creazione dell'ordine e verifica che tutti gli attributi (ID, IDuser, data, totale e lista degli item) siano corretti.
 - Test dei setter per modificare lo stato dell'ordine.

3.5. ProductTest

- **Obiettivo:** Assicurare che gli oggetti **Product** vengano creati correttamente tramite costruttore parametrizzato e tramite costruttore di default (con successiva impostazione tramite setter).
- **Casi di test:**
 - Verifica dell'inizializzazione dei campi (ID, size e modello associato).

3.6. SoldProductTest

- **Obiettivo:** Testare la corretta impostazione e lettura degli attributi di **SoldProduct**.
- **Casi di test:**
 - Impostazione di tutti i campi e verifica tramite i metodi getter.

3.7. UserTest

- **Obiettivo:** Validare che tutti gli attributi dell'utente (ID, nome, cognome, email, password, ruolo) siano correttamente gestiti.
- **Casi di test:**
 - Test completo dei metodi setter e getter per ogni attributo.

3.8. CartTest

- **Obiettivo:** Verificare la logica di gestione del carrello, in particolare:
 - L'aggiunta e la fusione degli item (incremento della quantità in caso di duplicati).
 - La rimozione di singoli item e di tutti gli item.
 - La corretta restituzione degli item presenti nel carrello.
- **Casi di test:**
 - Aggiunta di item nuovi e già esistenti.
 - Verifica del metodo di rimozione basato sull'indice e tramite predicato.
 - Test del reset del carrello.

3.9. CurrentTest

- **Obiettivo:** Assicurare la gestione corretta dello stato globale dell'applicazione, verificando:
 - L'impostazione e il recupero dell'utente corrente.
 - La gestione delle visualizzazioni (product e category view).
 - La corretta gestione del logout (ExitUser) che deve resettare lo stato e svuotare il carrello.
- **Casi di test:**
 - Verifica dei metodi statici per impostare e recuperare le proprietà di stato.

3.10. DatabaseManagerTest

- **Obiettivo:** Testare l'interazione con il database, verificando il corretto funzionamento di tutte le operazioni CRUD e delle query implementate.
- **Aspetti principali testati:**
 - **Connessione al database:** Verifica che il metodo getConnection() restituisca una connessione valida.
 - **Gestione degli utenti:** Salvataggio, aggiornamento e recupero degli utenti tramite saveUser(), updateUser(), findUserByID() e findUserByEmail().
 - **Gestione dei modelli e prodotti:** Salvataggio, recupero e cancellazione dei modelli (metodi come saveModel(), returnModelByName(), returnModelsOnCategory(), deleteModelByName()) e il corretto caricamento e conteggio dei prodotti.

- **Operazioni sugli indirizzi:** Salvataggio, aggiornamento, recupero e cancellazione degli indirizzi associati a un utente.
- **Gestione degli ordini e dei prodotti venduti:** Salvataggio degli ordini e dei prodotti venduti, e successivo recupero tramite `getOrdersByUserId()`, `getOrders()` e `getSoldProductsByOrderId()`.
- **Test di aggiornamento delle quantità:** Verifica che il metodo `updateQuantity()` aggiunga o rimuova il numero corretto di record in base alla richiesta.
- **Accorgimenti per il testing:**

Poiché la classe **DatabaseManager** opera sul database reale, è stata implementata una logica per registrare gli ID iniziali delle tabelle e, al termine dei test, cancellare solo le righe aggiunte durante l'esecuzione della suite. Questo approccio evita di interferire con eventuali dati preesistenti e garantisce l'isolamento dei test.

4. Copertura

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Code	19,2 %	3.639	15.315	18.954
src/main/java	11,2 %	1.924	15.277	17.201
com.example.progetto.GUI.areaprivata	0,0 %	0	3.434	3.434
com.example.progetto.GUI.dialogs	0,0 %	0	3.384	3.384
com.example.progetto.GUI.checkoutform	0,0 %	0	2.628	2.628
com.example.progetto.GUI.imagegallery	0,0 %	0	2.497	2.497
com.example.progetto.GUI.admin	0,0 %	0	1.035	1.035
com.example.progetto.GUI.Cart	0,0 %	0	779	779
com.example.progetto.GUI	0,0 %	0	535	535
com.example.progetto.GUI.recaporder	0,0 %	0	449	449
com.example.progetto.GUI.home	0,0 %	0	273	273
com.example.application.services	85,0 %	1.446	255	1.701
DatabaseManager.java	85,0 %	1.446	255	1.701
com.example.progetto	0,0 %	0	8	8
com.example.progetto.backend	100,0 %	478	0	478
Address.java	100,0 %	66	0	66
Cart.java	100,0 %	106	0	106
CartItem.java	100,0 %	23	0	23
Current.java	100,0 %	35	0	35
Model.java	100,0 %	77	0	77
Order.java	100,0 %	59	0	59
Product.java	100,0 %	36	0	36
SoldProduct.java	100,0 %	31	0	31
User.java	100,0 %	45	0	45
src/test/java	97,8 %	1.715	38	1.753
com.example.progetto.backend	97,0 %	995	31	1.026
com.example.application.services	99,0 %	720	7	727

La copertura complessiva del progetto può apparire bassa (19,2%) a causa dell'elevato numero di linee di codice dedicate all'interfaccia grafica GUI, la cui verifica non rientrava tra gli obiettivi del testing. Al contrario, l'intero backend, vero

fulcro dell'applicazione, gode di una copertura totale (100%): ciò conferma che gli obiettivi di test, incentrati sulla logica di business, sono stati pienamente raggiunti.

5. Conclusioni

La suite di test implementata copre in maniera esaustiva il funzionamento del backend del progetto, assicurando che:

- I dati vengano inizializzati e manipolati correttamente.
- Le operazioni CRUD sul database siano affidabili e gestiscano correttamente le eccezioni.
- La logica di business (come la gestione del carrello e dello stato globale) funzioni come previsto.
- Il testing della classe **DatabaseManager** garantisca la corretta integrazione con il database, con particolari accorgimenti per isolare le modifiche effettuate durante i test.